

University of Management and Technology Lahore
School of Science and Technology
Department of Computer Science

Complex Computing Problem (CCP)

Semester: Spring 2026
Course: Operating Systems (CC3011)
Maximum Marks: 100
Submission Date: 17th June, 2026

Marks Distribution

Section	CLOs	Marks
Problem understanding	CLO1	5
Race condition explanation	CLO2	5
Critical section identification	CLO2	5
Mutex lock explanation	CLO3	10
Thread pool explanation	CLO3	10
FCFS queue explanation	CLO2	5
Deadlock prevention	CLO3	10
OS concepts table	CLO1	5
Performance Analysis	CLO4	10
C/C++ code correctness	CLO3	20
Viva	CLO3	15
Total		100

Problem statement

A university online course registration system receives thousands of student requests during enrollment. Each course has limited seats. If multiple students try to register for the same course at the same time, a race condition may occur and two students may get the same last seat. Also, if a new thread is created for every request, the system may crash due to CPU and memory overload.

Design an operating system-based solution in C or C++ using **mutex locks**, **thread pool**, and **request queue and OS concepts**. Your solution should prevent race conditions, avoid server crash during high traffic, process requests fairly using FCFS scheduling, and prevent deadlock when students register for multiple courses. Assume that **5000 students** send registration requests at the same time. Provide performance analysis of course registration system by analyzing the given table.

The university tests the system using different thread pool sizes.

Case	Worker Threads	Total Requests	Average Response Time	CPU Usage	Memory Usage
A	50	5000	9 seconds	55%	Low
B	100	5000	5 seconds	75%	Medium
C	500	5000	4 seconds	95%	High
D	5000	5000	System crash	100%	Very High

Required Final Answer Format

Your document should include the following sections:

1. Introduction

Briefly explain the problem in the university course registration system.

2. Race Condition

Explain how two students may get the same last seat if synchronization is not used.

3. Critical Section

Identify the critical section in the registration process.

4. Mutex Lock Solution

Explain synchronization in registration process using mutex lock.

5. Thread Pool Solution

Explain how a fixed-size thread pool prevents system crash during high traffic.

6. Request Queue

Explain how requests are managed using FCFS scheduling.

7. Deadlock Prevention

Explain how fixed lock ordering prevents deadlock.

8. OS Concepts Used

Fill the following table:

Problem	OS Concept Used
Two students getting same seat	
High traffic	
Too many requests	
Fair registration	
Multiple course registration deadlock	

9. Performance Analysis of Course Registration System

Analyze and answer the following:

1. Which thread pool size is most suitable for this system and why?
2. Why does the system crash when 5000 threads are created?
3. How does the mutex lock affect system performance?
4. What is the trade-off between response time and resource usage?
5. Why is Case B better than Case C, even though Case C has slightly better response time?
6. Suggest two improvements to increase performance without causing system failure.

10. Implementation in C or C++